
Integración y Entrega Continua de Software Embebido

Caso de estudio: Jenkins

Daniel Chavarría García

Instituto Tecnológico de Costa Rica | Escuela de Ingeniería Electrónica

Taller de Sistemas Embebidos | Prof. Ing. Johan Carvajal Godínez

18 de septiembre de 2026



Introducción

El problema y por qué Jenkins

Un producto embebido moderno integra miles de cambios diarios de código sobre decenas de configuraciones de hardware. Validar manualmente esa combinación es inviable: el costo del laboratorio y la duración de cada ciclo se vuelven el cuello de botella del proyecto.

La dificultad

El dominio embebido depende del hardware, arrastra múltiples versiones simultáneas y no puede virtualizar su entorno de producción como sí lo hace la web.

La respuesta

Jenkins es un servidor de automatización *auto-alojado* y distribuido: permite conectar hardware físico a sus agentes y versionar el proceso de construcción junto con el código.

Objetivo: entender cómo se adapta el ciclo CI/CD a las restricciones físicas del software embebido.



Términos clave (1 de 2)

CI — Integración Continua Fusionar y verificar automáticamente cada cambio de código en un repositorio común.

CD — Entrega/Despliegue Continuo Automatizar el camino desde el control de versiones hasta el artefacto entregable o el dispositivo final.

Pipeline Secuencia automatizada de etapas (*stages*) que un cambio debe atravesar.

Cross-compilation Compilar en una máquina de arquitectura distinta a la del dispositivo destino (x86 → ARM).

Toolchain Conjunto de compilador, enlazador y bibliotecas dirigido a una arquitectura específica.

Artefacto Producto verificable del build: binario, imagen del sistema o paquete.



Términos clave (2 de 2)

Controller / Agente El controlador orquesta y aloja la interfaz; los agentes ejecutan el trabajo pesado.

Jenkinsfile Archivo de texto, versionado con el código, que define el pipeline completo.

HIL — Hardware in the Loop Ejecutar pruebas automatizadas sobre la tarjeta física real, no sobre un simulador.

On-target testing Pruebas que corren dentro del dispositivo destino, con su propio toolchain.

Regresión Defecto introducido por un cambio que rompe funcionalidad previamente correcta.

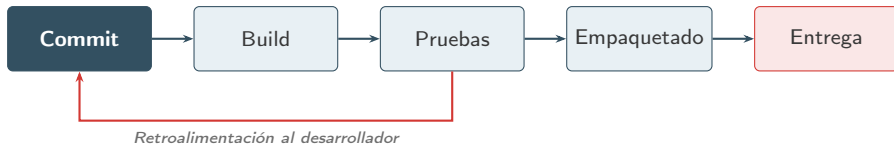
sstate-cache Caché de estado compartido de Yocto que reutiliza resultados de compilaciones previas.





Contexto

CI/CD: el ciclo de retroalimentación



Idea central

El valor de CI/CD no está en automatizar por automatizar, sino en **acortar el tiempo entre introducir un defecto y detectarlo**. Cuanto más tarde aparece un fallo, más caro resulta corregirlo.



El choque de realidades

	Dominio web	Sistemas embebidos
Ciclos	Despliegues diarios; iteraciones de dos semanas	Ciclos de liberación de diez meses a tres años
Equipos	Equipos de característica auto-organizados	Silos por módulo: hardware, firmware, sistema operativo
Entorno	Virtualización completa	Dependencia del hardware físico
Despliegue	Directo a producción, reversible	Específico por cliente, sin tolerancia a fallos
Monitoreo	Datos de uso y experimentación A/B	Ausencia de datos tras la entrega

Adaptado de Lwakatare et al. (2016), estudio de casos en cuatro empresas del sector embebido.



Los cuatro retos del dominio embebido

1. **Dependencia del hardware y compatibilidad entre versiones.** Los ciclos largos de desarrollo de hardware retrasan la liberación de software.
2. **Visibilidad limitada del entorno del cliente.** Cada instalación tiene su propia configuración, con elementos de otros proveedores.
3. **Falta de tecnología para desplegar de forma automática y repetible.** En sistemas críticos no se puede detener la máquina para actualizar.
4. **Ausencia de datos de uso.** Sin telemetría no hay realimentación para la mejora continua del producto.

Consecuencia de diseño

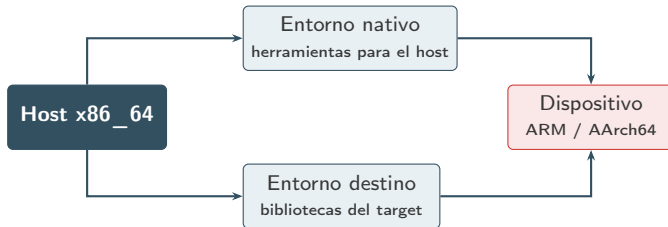
El pipeline embebido no puede ser una copia del pipeline web: debe asumir que el recurso escaso no es el CPU, sino **el hardware de prueba y el tiempo**.





El problema técnico

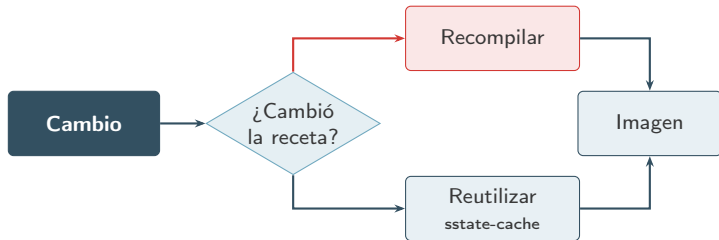
El costo de compilar: no es una aplicación, es un sistema



Compilar para un destino distinto exige mantener **dos sistemas de archivos raíz en paralelo**: uno con las herramientas del host y otro con las bibliotecas del dispositivo. Ese desdoblamiento multiplica el costo: un build limpio puede superar las **cuatro horas**, de modo que ejecutarlo en cada *commit* resulta inviable.



sstate-cache: reutilizar en lugar de recompilar



Yocto almacena el resultado de cada tarea. Si la entrada no cambió, la salida se recupera del caché en lugar de recompilarse. Un agente de CI con caché persistente convierte un build de horas en uno de minutos.

Por eso el agente de compilación debe ser una máquina dedicada y con estado, no un contenedor efímero.



Escalonamiento de pruebas: no todo corre siempre

Etapas	Alcance	Duración	Frecuencia
UT	Pruebas unitarias	Minutos	Por commit
BVT	Arranque e interfaces	< 20 min	Por commit
BAT	Regresión básica	5 – 8 h	Build diario
NTE	Regresión extendida	Varias noches	Nocturna
FAI	Inspección previa al release	—	Por versión

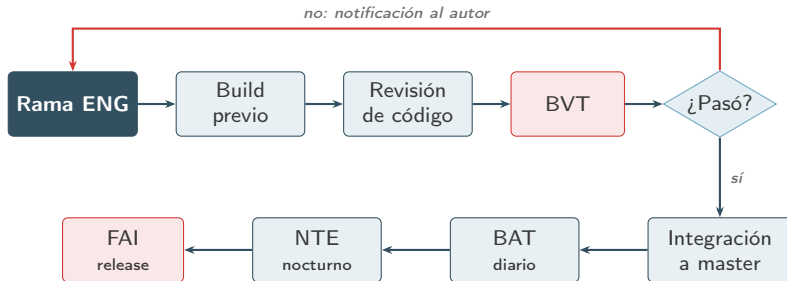
El principio de diseño

BVT se limita deliberadamente a menos de 20 minutos para que el desarrollador reciba una respuesta útil antes de cambiar de tarea. Las pruebas costosas se agrupan y se ejecutan cuando el hardware está libre.

Proceso reportado en la validación de sistemas embebidos complejos (IEEE ICSSP).



Flujo de validación industrial



Al detectarse una regresión, el desarrollador responsable dispone de un plazo acotado — **24 horas** en el caso documentado — para corregir o retirar su cambio.





Jenkins

Qué es Jenkins

Servidor de automatización de código abierto que observa un repositorio y ejecuta, ante cada cambio, la secuencia de etapas definida por el equipo.

Por qué en embebidos

- Auto-alojado: el hardware físico se conecta a sus propios agentes.
- Caché y estado persistentes en disco local.
- Opera en redes cerradas.

Costo asociado

- Requiere administrar el servidor.
- Ecosistema de complementos amplio, pero con carga de mantenimiento.

La documentación oficial recomienda la sintaxis declarativa de *pipeline* sobre los antiguos trabajos configurados por formulario.



Arquitectura: controlador y agentes

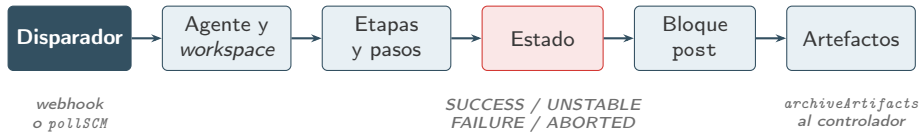


Regla práctica

Ningún trabajo debe ejecutarse en el controlador: separar la orquestación de la carga pesada evita que un build de horas bloquee la programación de tareas.



Ciclo de vida de una ejecución



Resiliencia del proceso

Jenkins persiste el estado de la ejecución en disco, de modo que un pipeline puede sobrevivir al reinicio del controlador o a la caída de un agente y reanudarse en el punto donde se interrumpió.

Docker y Jenkins: dos relaciones distintas

Jenkins *dentro* de un contenedor

Aísla el servidor de sus dependencias y lo hace reproducible.

Clave: usar un volumen nombrado para `/var/jenkins_home`. La documentación desaconseja los *bind mounts* por problemas de permisos.

Límite: nunca compilar dentro del controlador; un build de Yocto lo satura.

Contenedores *como* agentes

Empaquetan el *toolchain* exacto de cada etapa, sin instalarlo en la máquina física.

Clave: `agent { docker { image ... } }` en el `Jenkinsfile`.

Límite: el contenedor arranca con sistema de archivos limpio; hay que montar el caché de forma explícita.

Para flashear hardware real el contenedor necesita acceso privilegiado a los dispositivos: ahí el agente físico sigue siendo más simple.



Pipeline as Code: el Jenkinsfile

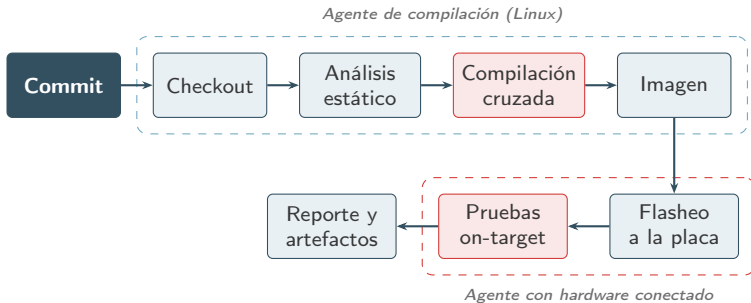
El proceso de construcción se versiona junto al código fuente: cada cambio del pipeline queda revisado y trazable.

```
pipeline {
  agent { label 'linux-build' }           // agente dedicado
  stages {
    stage('Build') {
      steps { sh 'make CROSS_COMPILE=arm-linux-gnueabihf-' }
    }
    stage('Verificar arquitectura') {      // etapa clave
      steps { sh 'file build/app | grep ARM' }
    }
    stage('Pruebas') { steps { sh 'make test' } }
  }
  post {
    success { archiveArtifacts artifacts: 'build/*' }
    failure { echo 'Build fallido: notificar al autor' }
  }
}
```

Verificar la arquitectura del binario distingue un pipeline embebido de uno convencional.



Diagrama de bloques del pipeline embebido



Las etapas rápidas y puramente software se ejecutan en el agente de compilación; solo las que requieren la tarjeta física ocupan el recurso escaso.



Diseño del pipeline: fases 1 y 2

Fase	Decisiones	Artefactos	Criterio de cierre
1. Diseño	Topología controlador/agentes; <i>toolchain</i> ; simulación frente a hardware real; estrategia de caché; niveles de prueba	Documento de arquitectura; imágenes con <i>toolchain</i> ; nodos conectados al banco	Arquitectura aprobada; conectividad verificada; credenciales definidas
2. Desarrollo	<i>Pipeline as Code</i> ; reglas de construcción; verificación de integridad del binario	Código fuente y recetas; Jenkinsfile versionado; pruebas unitarias y de verificación	Compilación cruzada exitosa; pruebas rápidas en verde; revisión de código aprobada

Riesgos propios del dominio

Fase 1: la escasez de hardware no virtualizable es el cuello de botella. Fase 2: el desarrollo en silos dificulta la integración de punta a punta.



Diseño del pipeline: fases 3 y 4

Fase	Decisiones	Artefactos	Criterio de cierre
3. Implementación y validación	Programación de las pruebas costosas; automatización del flasheo; bloqueo de placas como recurso exclusivo	Reportes de ejecución; trazas de diagnóstico; métricas del build	Sin regresiones en la regresión diaria; validación nocturna superada
4. Entrega	Entrega frente a despliegue continuo; firmado del firmware; gestión de artefactos	Binario firmado; imágenes del sistema; manifiesto de paquetes	Inspección final aprobada; firmas verificadas; publicación realizada

Por qué la entrega se detiene antes del despliegue

En sistemas críticos no se puede detener la máquina para actualizar, y un corte de energía durante la escritura puede inutilizar el dispositivo. El paso final queda bajo aprobación humana.



Jenkins frente a GitHub Actions

Dimensión	Jenkins	GitHub Actions
Alojamiento	Autoalojado, control total	Gestionado; <i>runners</i> propios opcionales
Definición	Jenkinsfile en Groovy	Flujo en YAML
Hardware físico	Agentes por USB/JTAG	<i>Runner</i> con conectividad saliente
Credenciales	Cifradas en el controlador	Secretos del repositorio, OIDC
Extensiones	Amplio ecosistema de complementos	Catálogo de acciones
Mantenimiento	Alto: servidor propio	Bajo: sin servidor central

Cuándo elige cada una

Jenkins: laboratorios aislados, *toolchains* pesados, caché en disco local. **GitHub Actions:** equipos pequeños con el código ya en GitHub y verificación en contenedores efímeros.



Control físico: dónde se decide la elección

	Jenkins (local)	GitHub Actions (nube)
Acceso al hardware	Interactúa con puertos serie y JTAG en la red del laboratorio	Acceso indirecto: requiere un <i>runner</i> instalado junto al equipo
Persistencia	Conserva el estado de las placas y el caché entre ejecuciones	Contenedores efímeros que se destruyen al terminar
Exposición	Controla instrumentación sin exponer la red a internet	Exige conectividad saliente constante hacia el servicio

El criterio de fondo

La diferencia no está en la calidad de la herramienta, sino en **dónde vive el recurso escaso**. Si el build necesita tocar hardware que no puede salir de la institución, la ejecución tiene que ocurrir donde está ese hardware.



Limitaciones y alternativas

Lo que CI/CD no resuelve en embebidos

El costo del laboratorio y del hardware de prueba sigue siendo el factor dominante, y la validación completa nunca cabe en el ciclo de un commit.

Herramienta	Fortaleza	Limitación en embebidos
Jenkins	Auto-alojado, agentes con hardware	Administración propia
GitLab CI	Integrado al repositorio	Requiere <i>runners</i> propios
GitHub Actions	Configuración sencilla	Orientado a la nube
Buildbot	Base del Autobuilder	Configuración en Python





Casos aplicados

Caso 1: el Autobuilder del Yocto Project

El propio Yocto Project mantiene una infraestructura de integración continua pública que construye y prueba cada versión sobre el hardware soportado.

Qué hace

- Varía MACHINE, DISTRO y opciones del sistema.
- Verifica que cada combinación compile.
- Ejecuta pruebas de rendimiento del build.

Lo relevante aquí

La capa que define qué pruebas correr es independiente del sistema de CI: puede operar con Buildbot, **con Jenkins** o con otros.

Toda la infraestructura y los resultados son públicos, de modo que otras organizaciones pueden reproducir el proceso.



Caso 2: aplicación al proyecto del curso

El proyecto *meta-acceso* construye una imagen Yocto para Raspberry Pi 4 con una aplicación de control de acceso en Python y GStreamer. Es un candidato natural para este pipeline.



Dónde se detiene la automatización

La tarjeta pertenece al laboratorio de la universidad, así que el último paso queda fuera del pipeline: la imagen se traslada y se graba en sitio. El cuello de botella no es el software, es el acceso al hardware.

El impacto medible

Métrica DORA	Qué mide	Resultado reportado
Frecuencia de despliegue	Cadencia de entrega	de ~30 a ~120 al mes
Tiempo de entrega del cambio	Del commit a producción	de 30+ a ~12 min
Tasa de fallo del cambio	Fiabilidad del release	—87 % en errores
Tiempo medio de recuperación	Reacción ante fallo	de 97 a 12 min

Lectura crítica

Estas cifras provienen de software empresarial, no del dominio embebido: dimensionan el orden de magnitud, pero **no son directamente extrapolables** a un producto con hardware en el lazo.





Cierre

Buenas prácticas para pipelines embebidos

Recomendado

- Usar sintaxis declarativa: más legible y uniforme entre proyectos.
- Delegar la lógica pesada a `make` o scripts nativos; el `Jenkinsfile` solo orquesta.
- Un solo ejecutor por agente con hardware conectado.
- Gestionar credenciales fuera del script.

Por evitar

- Ejecutar trabajos en el controlador.
- Comandos destructivos de limpieza en entornos paralelos: destruyen el caché compartido.
- Lógica compleja en el script del pipeline.
- Asumir que el build es reproducible sin fijar versiones del toolchain.



Conclusiones

1. El obstáculo para adoptar CI/CD en embebidos no es la herramienta, sino la **dependencia física del hardware** y la imposibilidad de virtualizar el entorno de producción.
2. La respuesta de la industria es **escalonar** la verificación: pruebas rápidas por commit, pruebas costosas de forma diferida.
3. Jenkins encaja en este dominio por ser auto-alojado y distribuido: permite dedicar agentes a la compilación pesada y otros al hardware real.
4. **Pipeline as Code** aporta trazabilidad, un requisito que va más allá de la conveniencia cuando existe certificación de por medio.

El hardware define los límites físicos; la automatización define la velocidad de iteración.



Referencias



L. E. Lwakatare et al. *Towards DevOps in the Embedded Systems Domain: Why is It so Hard?* HICSS, 2016.



Continuous Integration in Validation of Modern Complex Embedded Systems. IEEE ICSSP.



C. C. Dias et al. *On-Target Test Automation for an Embedded Digital Signal Processing Device*. IEEE ICCE, 2022.



S. Mysari y V. Bejgam. *Continuous Integration and Continuous Deployment Pipeline Automation Using Jenkins Ansible*. ic-ETITE, 2020.



S. P. Subramanyam. *DevOps and CI/CD Maturity in Large-Scale Organizations: A SonarQube and Jenkins Approach*. IEEE ICAIC, 2026.



Jenkins Project. *User Handbook: Pipeline*. <https://www.jenkins.io/doc/book/pipeline/>



Yocto Project. *Test Environment Manual*. <https://docs.yoctoproject.org/test-manual/intro.html>



¡Gracias!

Preguntas y comentarios



Jenkins